AI BlogNest

Project Testing Phase

1. Introduction

Testing is an important phase in the development of AI BlogNest. The main purpose of testing is to verify that the application works correctly, satisfies the functional requirements, and provides a reliable experience for users. The testing process identifies errors, unexpected behaviour, interface problems, and integration issues before the project is delivered.

2. Objectives of Testing

The major objectives are to verify every important feature of AI BlogNest, identify and correct defects, check the interaction between frontend and backend components, validate data handling, confirm that AI-related features respond appropriately, and ensure that the application is usable and stable under normal operating conditions.

3. Testing Strategy

A systematic testing strategy is followed for AI BlogNest. Individual modules are tested first, followed by integration testing and complete system testing. Test cases are prepared from the project requirements. Each test case contains an input, expected result, actual result, and final status. Failed cases are analysed, corrected, and tested again.

4. Unit Testing

Unit testing checks individual components independently. In AI BlogNest, important units include user input handling, blog creation functions, login or registration validation, database operations, content processing, and AI-assisted functionality. The purpose is to ensure that each component performs its intended task correctly before it is combined with other components.

5. Integration Testing

Integration testing verifies communication between connected modules. For example, the user interface must correctly communicate with the backend, the backend must process requests correctly, and database operations must store and retrieve information accurately. AI services, when used, are also checked to ensure that requests and responses are transferred correctly.

6. Functional Testing

Functional testing checks whether the system performs the functions defined in the requirements. Typical test cases include opening the application, registering a user, logging in, creating a blog, editing content, viewing blog posts, searching or filtering content, saving information, and logging out. The expected output is compared with the actual output for every case.

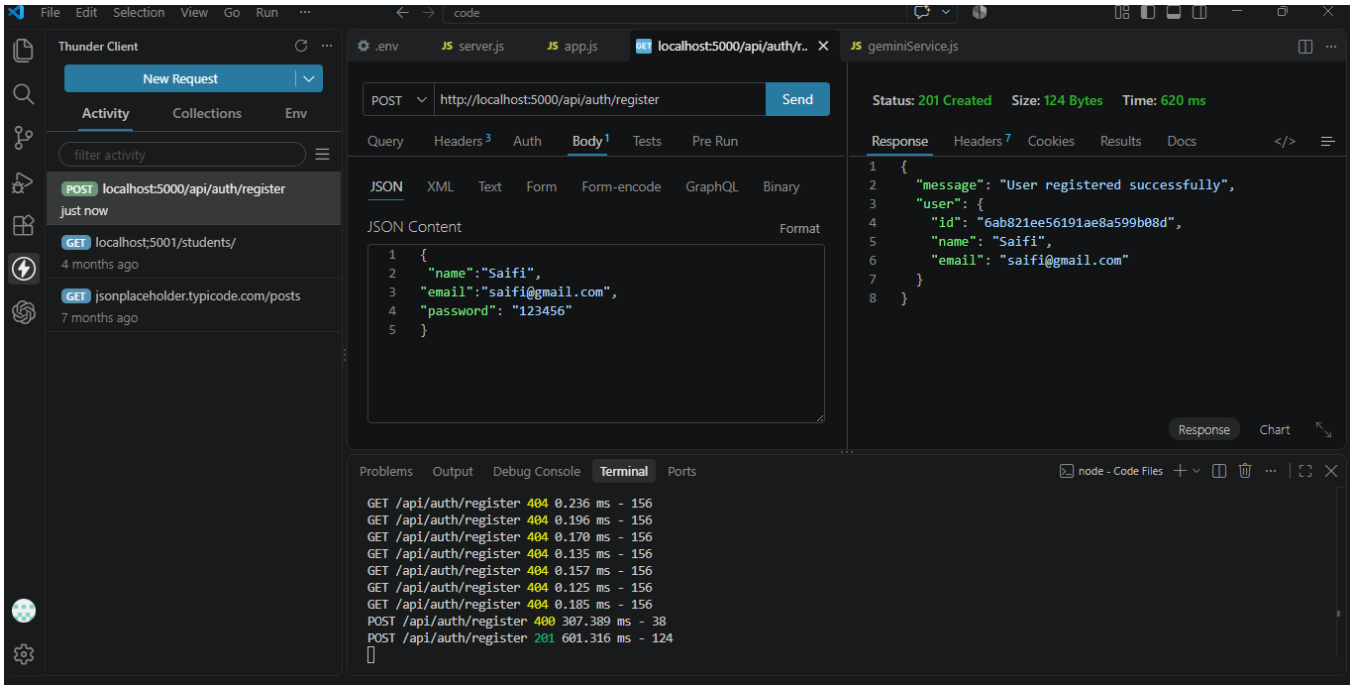


Figure 1: User registration – POST /api/auth/register returns 201 Created.

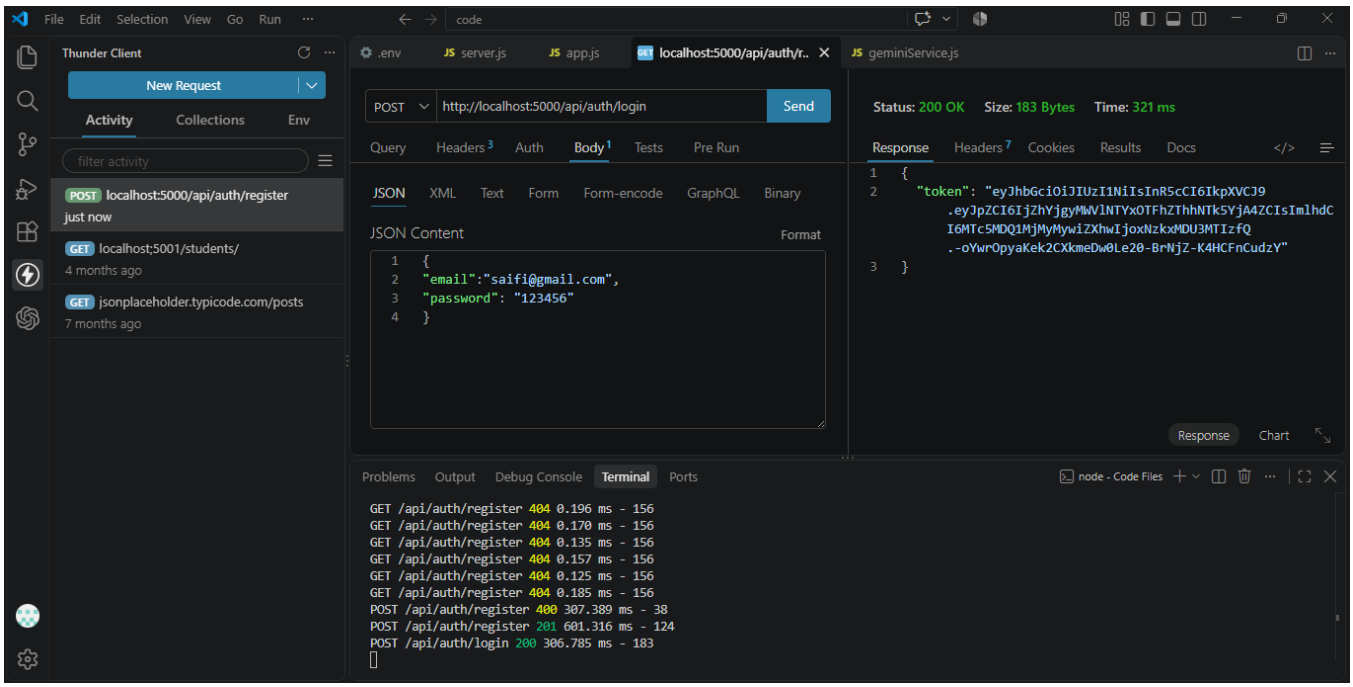


Figure 2: User login – POST /api/auth/login returns 200 OK and a JWT token.

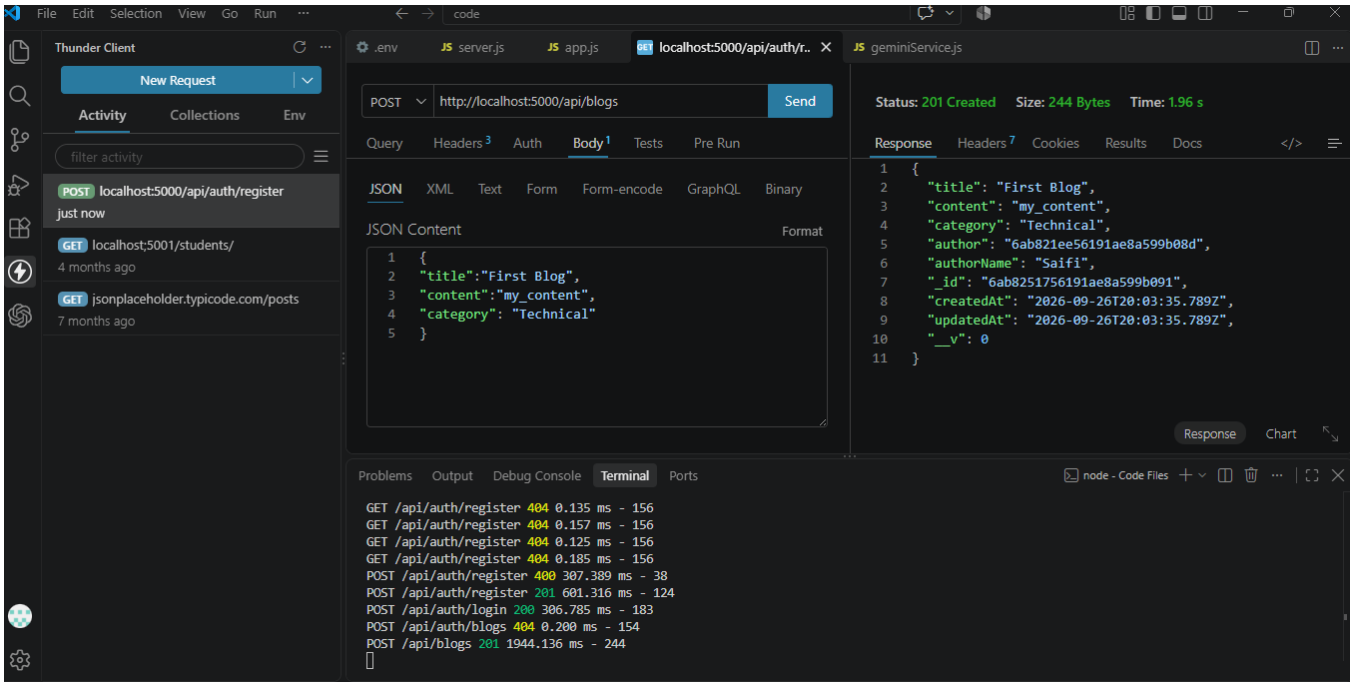


Figure 3: Blog creation – POST /api/blogs returns 201 Created.

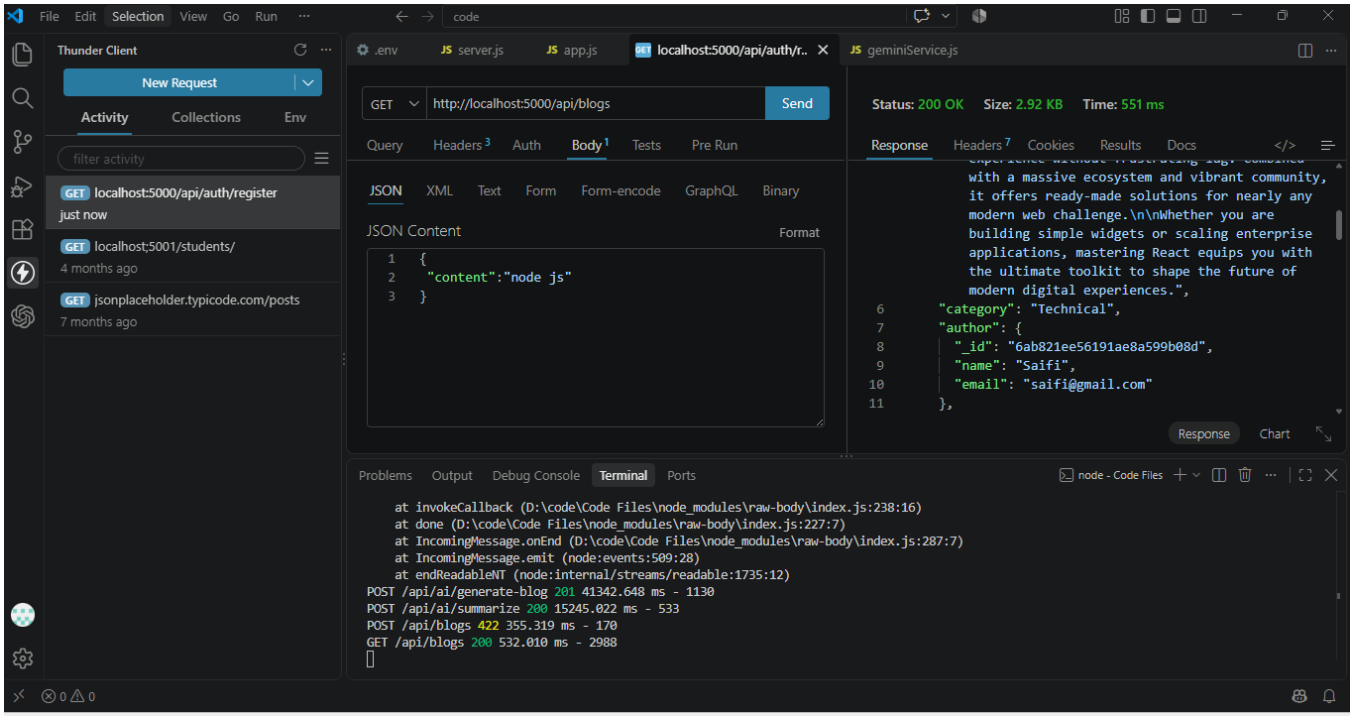


Figure 4: Blog retrieval – GET /api/blogs returns the stored blog data.

7. Usability Testing

Usability testing focuses on how easily users can understand and operate AI BlogNest. Navigation, button labels, form fields, page layout, readability, error messages, and overall user flow are checked. The interface should allow users to complete common tasks without unnecessary confusion.

8. Performance Testing

Performance testing checks how the application behaves with normal and increased usage. Page loading, response time, database operations, and AI-related requests are observed. The system should provide responses within a reasonable time and should avoid unnecessary delays or failures.

9. Security Testing

Security testing checks whether user information and application data are handled safely. Input validation, authentication, access control, session handling, and protection against common invalid inputs are considered. Sensitive information should not be exposed through the interface or application responses.

10. Test Case Summary

Test cases are created for major project functions. A successful test is marked as PASS when the actual result matches the expected result. A test is marked as FAIL when the expected behaviour is not achieved. Failed cases are corrected and executed again until the required behaviour is obtained.

11. Bug Identification and Correction

During testing, defects may be identified in the user interface, application logic, database connectivity, validation, or AI integration. Each defect is recorded, analysed, corrected, and retested. Regression testing is then performed to ensure that a correction has not introduced a new problem in another part of the application.

AI Feature Testing Evidence

The following screenshots document the AI-assisted functions demonstrated during testing.

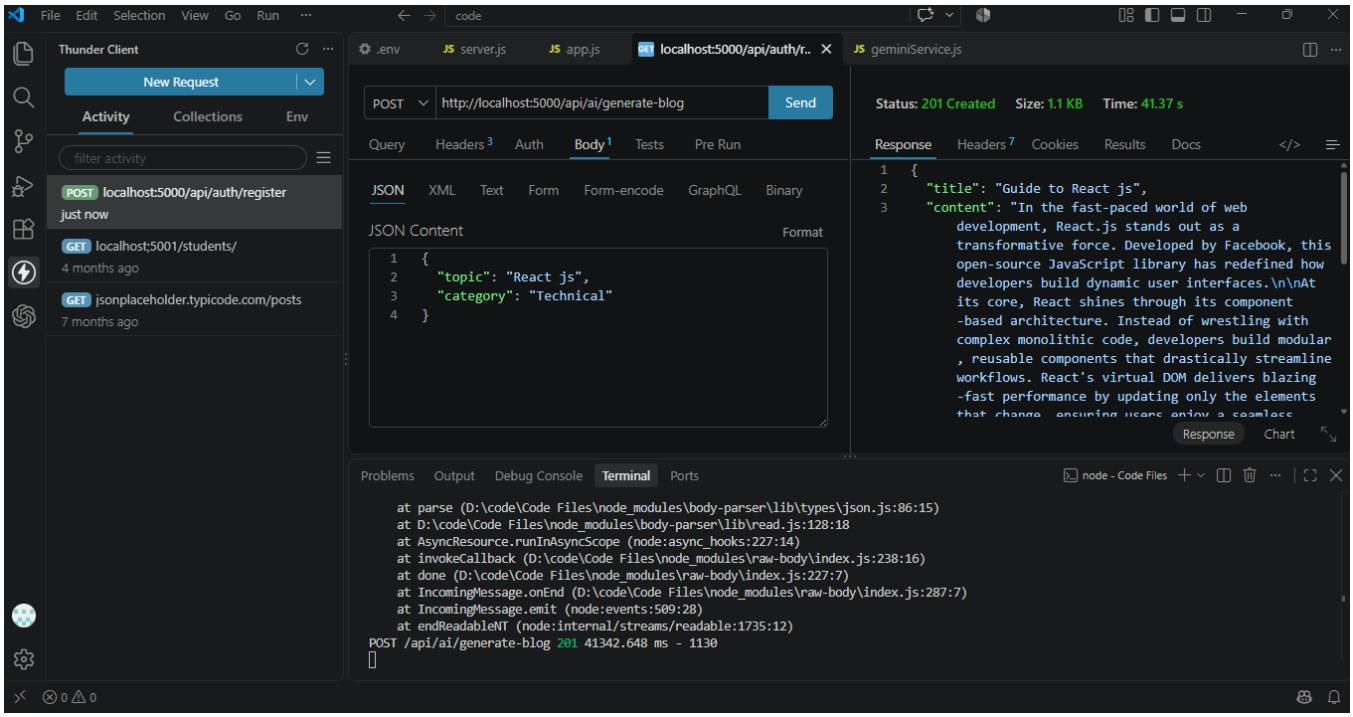


Figure 5: AI blog generation – POST /api/ai/generate-blog returns 201 Created.

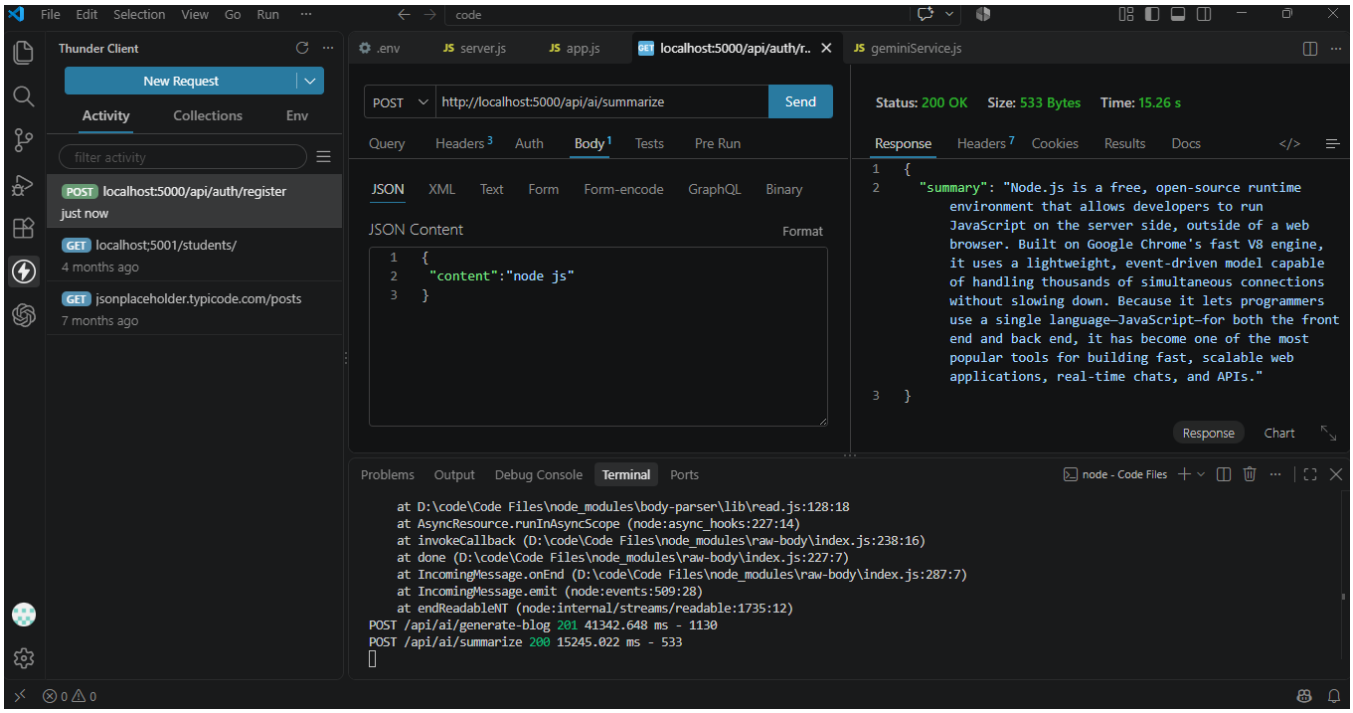


Figure 6: AI summarization – POST /api/ai/summarize returns 200 OK.

12. Final Testing Result

After completing the major testing activities, AI BlogNest is checked as an integrated application. The final testing confirms whether the important requirements are satisfied and whether the application is ready for demonstration or deployment. Testing therefore provides confidence that the developed system functions according to its intended design.